In [ ]:
```python
import random
from itertools import product
from pathlib import Path
from time import perf_counter

import pandas as pd
from pysmt.shortcuts import (
    FALSE, TRUE, And, Equals, EqualsOrIff, Iff, Int, Ite, Not, Or, Plus,
Solver, Symbol,
)
from pysmt.typing import BOOL

# ---------------------------------------------------------------------
# Parâmetros do problema
# ---------------------------------------------------------------------
N = 17  # dimensão da grelha de células normais (o enunciado exige N > 16)

# Relação de transição usada pela FSM (exigido pelo enunciado):
#   "assincrona" -> em cada transição só uma célula normal muda de estado
#   "sincrona"   -> em cada transição todas as células normais mudam em
simultâneo
MODO_TRANSICAO = "sincrona"
MODOS = ("assincrona", "sincrona")

SEMENTE = 0  # semente para sortear as células iniciais (resultados
reproduzíveis)
K_MAX = 10  # número máximo de estados dos traços explorados pelo BMC
SOLVER = "z3"
```

In [ ]:
```python
import marimo as mo
```

Exercício 1 — Game of Life como máquina de estados finita

1. Descrição do problema e abordagem

> **Objetivo** — modelar uma variante finita do *Game of Life* (grelha $N \times N$, $N > 16$, regra B3/S23) como uma FSM com duas relações de transição (síncrona e assíncrona) e verificar duas propriedades:
>
> 1. nunca se alcança o estado de erro (todas as células normais mortas);
> 2. nenhuma célula normal fica permanentemente morta ou viva.

Estratégia. O autómato é um FOTS (`declare` , `init` , `trans`), como na ficha 7, e as propriedades são verificadas com BMC através de duas funções genéricas, `bmc_always` ($G\,\phi$) e `bmc_eventually` ($F\,\phi$).

2. Escolhas

- **Ferramenta:** pySMT com Z3, a ferramenta da cadeira para BMC.
- **Estado:** uma variável booleana por célula normal (N^2). As células iniciais e finais são constantes e só entram na contagem de vizinhos.
- **Células iniciais:** sorteadas uma vez com `SEMENTE` e fixas; os resultados valem para essa moldura.
- **Síncrona:** todas as células tomam o valor da regra ao mesmo tempo.
- **Assíncrona:** o solver escolhe uma célula que possa mudar; se nenhuma puder, o estado transita para si próprio.
- **Propriedade 1:** invariante $G\,\neg\mathit{erro}$, com `bmc_always`.
- **Propriedade 2:** por célula, $F\,(c=1)$ e $F\,(c=0)$, com `bmc_eventually`; o contra-exemplo é um traço com lacete.
- **Limite:** sem contra-exemplo até `K_MAX`, o resultado só vale para traços desse tamanho.

3. Geometria da grelha

A grelha completa tem índices em $\{0..N+1\}^2$: as N^2 células normais e uma moldura de $(N+2)^2 - N^2 = 4N+4$ células.

Decisão sobre a moldura. O enunciado pede $2N+1$ células iniciais («um dos índices é zero») e $2N+3$ células finais («não são iniciais e um dos índices é $N+1$»). As duas contagens só batem certo se os índices das células iniciais estiverem em $\{0..N\}$:

- **iniciais:** $\{(i,j) \in \{0..N\}^2 : i = 0 \lor j = 0\}$, ou seja, $(0,0)$, $(0,1..N)$ e $(1..N, 0)$: $1 + N + N = 2N+1$ células;
- **finais:** as restantes células da moldura, todas com um índice igual a $N+1$: $(N+1, 0..N+1)$ e $(0..N, N+1)$: $(N+2) + (N+1) = 2N+3$ células.

Os cantos $(0, N+1)$ e $(N+1, 0)$ têm um índice igual a 0, mas contam como **finais**, porque o outro índice está fora de $\{0..N\}$. Só assim se obtém $(2N+1) + (2N+3) = 4N+4$, a moldura completa. A escolha tem efeito: estes cantos são vizinhos das células normais $(1, N)$ e $(N, 1)$, que passam a ver ali uma célula sempre morta, em vez de uma sorteada.

In [ ]:
```python
def celulas_normais(n=N):
    """Lista das N×N células normais (i, j), com i, j em 1..n."""
    celulas = []
    for i in range(1, n + 1):  # range exclui o fim, por isso vai até n + 1
        for j in range(1, n + 1):
            celulas.append((i, j))
    return celulas
```

In [ ]:
```python
def celulas_iniciais(n=N):
    """Lista das 2n+1 células iniciais: (i, j) em {0..n}² com i = 0 ou j = 0."""
    celulas = []
    for i in range(0, n + 1):
        for j in range(0, n + 1):
            if i == 0 or j == 0:  # o "ou" conta (0, 0) uma só vez
                celulas.append((i, j))
    return celulas
```

In [ ]:
```python
def celulas_finais(n=N):
    """Lista das 2n+3 células finais: as restantes da moldura,
      incluindo os cantos (0, n+1) e (n+1, 0)."""
    iniciais = celulas_iniciais(n)
    celulas = []
    for i in range(0, n + 2):
        for j in range(0, n + 2):
            if (i == n + 1 or j == n + 1) and (i, j) not in iniciais:
                celulas.append((i, j))
    return celulas
```

In [ ]:
```python
def vizinhos(i, j, n=N):
    """Lista dos 8 vizinhos da célula normal (i, j): as células (a, b) com
    max(|i - a|, |j - b|) = 1. Só aceita células normais, porque só estas mudam
    de estado; assim os vizinhos caem sempre dentro da moldura {0..n+1}².""""
    if not (1 <= i <= n and 1 <= j <= n):
        raise ValueError(f"({i}, {j}) não é uma célula normal para n = {n}")
    resultado = []
    for a, b in product(range(i - 1, i + 2), range(j - 1, j + 2)):
        if max(abs(i - a), abs(j - b)) == 1:  # a própria célula dá 0 e fica de fora
            resultado.append((a, b))
    return resultado
```

In [ ]:
```python
def sortear_iniciais(n=N, semente=SEMENTE):
    """Dicionário célula inicial -> 0/1, sorteado com probabilidade 1/2 e semente fixa."""
    rng = random.Random(semente)  # gerador local: não depende do random global
    valores = {}
    for celula in celulas_iniciais(n):  # lista com ordem fixa -> resultado reproduzível
        valores[celula] = int(rng.random() < 0.5)  # probabilidade 1/2; int em vez de bool
    return valores
```

In [ ]:
```python
def construir_fronteira(n=N, semente=SEMENTE):
    """Dicionário com o valor fixo de todas as células da moldura: iniciais sorteadas e finais a 0."""
    fronteira = dict(sortear_iniciais(n, semente))  # cópia: dicionário novo
```

```
    for celula in celulas_finais(n):
        fronteira[celula] = 0  # as células finais estão sempre mortas
    return fronteira
```

4. Estados e relações de transição

`declarar_estado(k)` cria a k-ésima cópia das variáveis (uma por célula normal). `init` impõe o estado inicial: viva se e só se $i, j \le 3$. `proximo_valor` dá, como fórmula, o valor de uma célula depois de aplicar B3/S23. A partir dela escrevem-se `trans_sincrona` e `trans_assincrona`, e `trans` escolhe uma delas pelo `modo`. `erro` é a conjunção de todas as células normais mortas.

Como o BMC é genérico, recebe `declare(i)`, `init(s)` e `trans(s, p)` só com os estados como argumentos. `fabricar_fsm` fixa a moldura, o modo e N e devolve essas três funções prontas a passar ao BMC.

In [ ]:
```
def declarar_estado(k, n=N):
    """Cria as variáveis do estado número k do traço; devolve um dicionário célula -> variável."""
    estado = {}
    for (i, j) in celulas_normais(n):
        # separadores no nome evitam colisões, ex.: (1, 11) e (11, 1) no mesmo estado
        estado[(i, j)] = Symbol(f"c_{i}_{j}_{k}", BOOL)
    return estado
```

In [ ]:
```
def valor_celula(s, celula, fronteira):
    """Termo/valor de uma célula no estado s: variável se for normal, constante se for inicial ou final."""
    if celula in s:  # célula normal: o valor é decidido pelo solver
        return s[celula]
    if celula in fronteira:  # célula da moldura: constante booleana
        return TRUE() if fronteira[celula] == 1 else FALSE()
    raise ValueError(f"{celula} não pertence à grelha")
```

In [ ]:
```
def init(s, fronteira, n=N):
    """Fórmula que caracteriza o estado inicial."""
    condicoes = []
    for (i, j) in celulas_normais(n):
        if i <= 3 and j <= 3:  # bloco 3×3 do canto: viva
            condicoes.append(s[(i, j)])
        else:  # todas as outras células normais: morta
            condicoes.append(Not(s[(i, j)]))
    return And(condicoes)
```

In [ ]:
```
def proximo_valor(s, celula, fronteira, n=N):
    """Fórmula do valor seguinte de uma célula normal segundo a regra B3/S23."""
    i, j = celula
    termos = []
    vivos_moldura = 0
```

```
    for v in vizinhos(i, j, n):
        if v in s:  # vizinha normal: conta 1 se a variável for verdadeira
            termos.append(Ite(valor_celula(s, v, fronteira), Int(1),
Int(0)))
        else:  # vizinha da moldura: valor constante, somado logo em Python
            vivos_moldura += fronteira[v]
    termos.append(Int(vivos_moldura))
    contagem = Plus(termos)
    # B3/S23 na forma compacta: viva sse 3 vizinhos, ou viva e 2 vizinhos
    return Or(Equals(contagem, Int(3)), And(s[celula], Equals(contagem,
Int(2))))
```

In [ ]:
```
def trans_sincrona(s, p, fronteira, n=N):
    """Relação de transição em que todas as células normais transitam em
simultâneo."""
    # cada célula de p toma o valor da regra calculado só a partir de s
    return And([Iff(p[c], proximo_valor(s, c, fronteira, n)) for c in
celulas_normais(n)])
```

In [ ]:
```
def trans_assincrona(s, p, fronteira, n=N):
    """Relação de transição em que uma e só uma célula normal transita de
estado (lacete se nenhuma puder mudar)."""
    restricoes = []
    mudancas = []  # termos 1/0 para contar as células que mudaram
    podem = []  # condições «a célula pode mudar»
    for c in celulas_normais(n):
        pode_mudar = Not(Iff(proximo_valor(s, c, fronteira, n), s[c]))
        mudou = Not(Iff(p[c], s[c]))
        restricoes.append(Or(Not(mudou), pode_mudar))  # mudou =>
pode_mudar
        mudancas.append(Ite(mudou, Int(1), Int(0)))
        podem.append(pode_mudar)
    # muda exatamente uma se alguma puder mudar; nenhuma se o estado for
estável
    restricoes.append(Equals(Plus(mudancas), Ite(Or(podem), Int(1),
Int(0))))
    return And(restricoes)
```

In [ ]:
```
def trans(s, p, fronteira, modo=MODO_TRANSICAO, n=N):
    """Escolhe a relação de transição de acordo com o parâmetro `modo`."""
    if modo == "sincrona":
        return trans_sincrona(s, p, fronteira, n)
    if modo == "assincrona":
        return trans_assincrona(s, p, fronteira, n)
    raise ValueError(f"modo inválido: {modo!r}; valores aceites: {MODOS}")
```

In [ ]:
```
def erro(s, n=N):
    """Fórmula do estado de erro: todas as células normais mortas.
    Não recebe fronteira: o enunciado define o erro só com as células
normais."""
    return And([Not(s[c]) for c in celulas_normais(n)])
```

In [ ]:
```
def fabricar_fsm(fronteira, modo=MODO_TRANSICAO, n=N):
    """Devolve (declare, init, trans) só com estados como argumentos,
prontos para o BMC genérico."""
```

```
    if modo not in MODOS:  # o erro aparece já aqui, e não a meio do BMC
        raise ValueError(f"modo inválido: {modo!r}; valores aceites:
{MODOS}")

    # nomes diferentes de init/trans, para chamarem as funções globais e
não a si próprias
    def declare_fsm(k):
        return declarar_estado(k, n)

    def init_fsm(s):
        return init(s, fronteira, n)

    def trans_fsm(s, p):
        return trans(s, p, fronteira, modo, n)

    return declare_fsm, init_fsm, trans_fsm
```

5. Simulação concreta (validação do modelo)

Simulador em Python puro, independente do solver, com dois papéis:

- mostrar a evolução da grelha a partir do estado inicial, em qualquer dos modos, para se perceber o comportamento antes de usar o BMC;
- confirmar que cada traço ou contra-exemplo devolvido pelo solver é uma execução válida da FSM: o primeiro estado é o inicial, e cada par de estados consecutivos respeita a regra no modo escolhido. Isto valida o modelo simbólico de forma independente.

In [ ]:
```
def estado_inicial_concreto(n=N):
    """Grelha inicial concreta (dicionário célula normal -> 0/1)."""
    grelha = {}
    for (i, j) in celulas_normais(n):
        grelha[(i, j)] = 1 if i <= 3 and j <= 3 else 0  # bloco 3×3 vivo
    return grelha
```

In [ ]:
```
def aplicar_regra(grelha, fronteira, n=N):
    """Dicionário célula normal -> valor que a regra B3/S23 lhe dá,
calculado a partir de grelha."""
    resultado = {}
    for celula in celulas_normais(n):
        vivos = 0
        for v in vizinhos(*celula, n):
            vivos += grelha[v] if v in grelha else fronteira[v]  # normal:
grelha; moldura: fronteira
        if grelha[celula] == 1:
            resultado[celula] = 1 if vivos in (2, 3) else 0  #
sobrevivência
        else:
            resultado[celula] = 1 if vivos == 3 else 0  # nascimento
    return resultado
```

In [ ]:
```python
def passo_concreto(grelha, fronteira, modo=MODO_TRANSICAO, rng=None, n=N):
    """Aplica uma transição concreta à grelha (no modo assíncrono, a célula que muda é escolhida por rng)."""
    if modo not in MODOS:
        raise ValueError(f"modo inválido: {modo!r}; valores aceites: {MODOS}")
    regra = aplicar_regra(grelha, fronteira, n)  # dicionário novo; a grelha não é alterada

    if modo == "sincrona":
        return regra

    if rng is None:
        raise ValueError("o modo assíncrono precisa de um gerador rng, ex.: random.Random(semente)")
    instaveis = [c for c in celulas_normais(n) if regra[c] != grelha[c]]
    nova = dict(grelha)
    if instaveis:  # se nenhuma for instável, o estado repete-se (lacete)
        c = rng.choice(instaveis)
        nova[c] = regra[c]
    return nova
```

In [ ]:
```python
def traco_e_valido(traco, fronteira, modo=MODO_TRANSICAO, n=N):
    """Verifica se uma sequência de grelhas é uma execução válida da FSM; devolve a lista de erros
    (lista vazia = traço válido). Aceita valores 0/1 ou True/False.
    Para validar um lacete, quem chama acrescenta ao fim do traço uma cópia do estado onde o
    lacete começa; assim a transição de regresso também é verificada."""
    if modo not in MODOS:
        raise ValueError(f"modo inválido: {modo!r}; valores aceites: {MODOS}")
    if not traco:
        return ["traço vazio: não há execução"]

    erros = []
    normais = celulas_normais(n)

    # Passo 1 — normalizar para 0/1 e verificar o formato de cada grelha
    grelhas = []
    for k, g in enumerate(traco):
        if set(g) != set(normais):
            erros.append(f"estado {k}: as chaves não são as células normais de uma grelha {n}×{n}")
            return erros
        g = {c: int(g[c]) for c in normais}
        maus = [c for c in normais if g[c] not in (0, 1)]
        if maus:
            erros.append(f"estado {k}: valores fora de {{0, 1}} em {maus}")
            return erros
        grelhas.append(g)

    # Passo 2 — o primeiro estado é o inicial
    inicial = estado_inicial_concreto(n)
    diferentes = [c for c in normais if grelhas[0][c] != inicial[c]]
```

```
    if diferentes:
        erros.append(f"estado 0: não é o estado inicial; diferem as células
{diferentes}")

    # Passo 3 — cada par consecutivo respeita a transição
    for k in range(len(grelhas) - 1):
        a, b = grelhas[k], grelhas[k + 1]
        regra = aplicar_regra(a, fronteira, n)
        if modo == "sincrona":
            for c in normais:
                if b[c] != regra[c]:
                    erros.append(f"passo {k}->{k + 1}: célula {c} devia
valer {regra[c]}, vale {b[c]}")
        else:
            instaveis = [c for c in normais if regra[c] != a[c]]
            mudaram = [c for c in normais if b[c] != a[c]]
            if not instaveis and mudaram:
                erros.append(f"passo {k}->{k + 1}: estado estável, mas
mudaram {mudaram}")
            elif instaveis and len(mudaram) != 1:
                erros.append(f"passo {k}->{k + 1}: devia mudar exatamente
uma célula instável, "
                             f"mudaram {len(mudaram)}: {mudaram}")
            elif instaveis and mudaram[0] not in instaveis:
                erros.append(f"passo {k}->{k + 1}: a célula {mudaram[0]}
mudou, mas não era instável")
    return erros
```

In [ ]:
```
def apresentar_grelha(grelha, fronteira=None, n=N):
    """Tabela (pandas) da grelha para mostrar no Marimo, com ou sem a
moldura de células iniciais/finais.
    Legenda: ■ viva e · morta nas células normais; ● viva e ○ morta na
moldura."""
    indices = range(1, n + 1) if fronteira is None else range(0, n + 2)
    linhas = []
    for i in indices:
        linha = []
        for j in indices:
            if (i, j) in grelha:
                linha.append("■" if int(grelha[(i, j)]) == 1 else "·")
            else:
                linha.append("●" if int(fronteira[(i, j)]) == 1 else "○")
        linhas.append(linha)
    return pd.DataFrame(linhas, index=list(indices), columns=list(indices))
```

In [ ]:
```
PASSOS_SIMULACAO = 200  # máximo do slider; o assíncrono muda uma célula
por passo

modo_sim = mo.ui.dropdown(
    options=list(MODOS), value=MODO_TRANSICAO, label="Modo de transição"
)
passo_sim = mo.ui.slider(
    start=0, stop=PASSOS_SIMULACAO, step=1, value=0, label="Passo",
show_value=True
)
```

```
semente_escalonador = mo.ui.number(
    start=0, stop=1_000_000, step=1, value=0,
    label="Semente do escalonador (só assíncrono)",
)
mostrar_moldura = mo.ui.checkbox(value=True, label="Mostrar moldura")

mo.vstack([
    mo.md("### Simulação concreta"),
    mo.hstack([modo_sim, semente_escalonador, mostrar_moldura],
justify="start"),
    passo_sim,
])
```

Simulação concreta

Modo de transição sincrona Semente do escalonador (só assíncrono) 0 Mostrar moldura

Passo 0

In [ ]:
```
_rng = random.Random(int(semente_escalonador.value or 0))  # um só gerador
para o traço todo
fronteira_sim = construir_fronteira(N, SEMENTE)  # a mesma moldura do resto
do trabalho
traco_sim = [estado_inicial_concreto(N)]
for _ in range(PASSOS_SIMULACAO):
    traco_sim.append(passo_concreto(traco_sim[-1], fronteira_sim,
modo_sim.value, _rng, N))
erros_sim = traco_e_valido(traco_sim, fronteira_sim, modo_sim.value, N)
```

In [ ]:
```
_k = passo_sim.value
_grelha = traco_sim[_k]
_vivas = sum(_grelha.values())

_linhas = [f"**Passo {_k}** de {PASSOS_SIMULACAO} · modo **
{modo_sim.value}** · "
           f"células vivas: **{_vivas}**"]
if _vivas == 0:
    _linhas.append("⚠️ **Estado de erro:** todas as células normais estão
mortas.")
if _k > 0:
    _mudaram = [c for c in _grelha if _grelha[c] != traco_sim[_k - 1][c]]
    if not _mudaram:
        _linhas.append("Estado igual ao anterior (estável).")
    elif modo_sim.value == "assincrona":
        _linhas.append(f"Mudou a célula {_mudaram[0]}.")
    else:
        _linhas.append(f"Mudaram {len(_mudaram)} células.")
_linhas.append("✅ Traço válido." if not erros_sim
               else f"❌ Traço inválido: {len(erros_sim)} erros (ex.:
{erros_sim[0]})")
_linhas.append("Legenda: ■ viva, · morta (células normais); ● viva, ○ morta
(moldura).")

_tabela = apresentar_grelha(_grelha, fronteira_sim if mostrar_moldura.value
else None, N)
mo.vstack([mo.md("\n\n".join(_linhas)), mo.Html(_tabela.to_html())])
```

Passo 0 de 200 · modo **sincrona** · células vivas: **9** ✅ Traço válido. Legenda: ■ viva, · morta (células normais); ● viva, ○ morta (moldura).

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	○	○	●	●	○	●	○	●	●	○	○	○	●	○	○	●	○	○	○
1	○	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
2	○	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
3	●	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
4	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
5	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
6	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
7	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
8	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
9	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
10	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
11	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
12	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
13	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
14	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
15	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
16	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
17	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
18	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

6. Bounded model checking genérico

As duas funções servem para qualquer FOTS: recebem `declare`, `init`, `trans`, a propriedade e o limite `K`, e não dependem do autómato.

- `bmc_always` (invariantes $G\,\phi$): para $k = 1..K$, procura um traço de k estados que começa num estado inicial, respeita `trans` e cujo último estado viola ϕ. Como k cresce, o primeiro contra-exemplo encontrado é mínimo.
- `bmc_eventually` (animação $F\,\phi$): para $k = 1..K$, procura um traço de k estados em que ϕ é falsa em todos os estados e o último estado transita para um dos anteriores (lacete). Isso é uma execução infinita em que ϕ nunca acontece. Devolve também o estado onde o lacete começa.

Ambas devolvem um dicionário com o resultado (`"contra-exemplo"` ou `"sem contra-exemplo até K"`), o número de estados, o traço (valores das variáveis em cada estado) e o tempo.

In [ ]:
```python
def extrair_traco(modelo, estados):
    """Converte o modelo do solver numa lista de dicionários (variável -> valor Python), um por estado."""
    traco = []
    for estado in estados:
        traco.append({chave: modelo.get_py_value(var) for chave, var in estado.items()})
    return traco
```

In [ ]:
```python
def bmc_always(declare, init, trans, inv, K):
    """BMC genérico de um invariante G inv: devolve resultado, k, traço (contra-exemplo mínimo, se houver) e tempo."""
    inicio = perf_counter()
    with Solver(name=SOLVER) as solver:
        estados = [declare(0)]
        solver.add_assertion(init(estados[0]))  # permanente
        for k in range(1, K + 1):
            if k > 1:
                estados.append(declare(k - 1))
                solver.add_assertion(trans(estados[-2], estados[-1]))  # permanente
            solver.push()
            solver.add_assertion(Not(inv(estados[-1])))  # só para este k
            if solver.solve():
                traco = extrair_traco(solver.get_model(), estados)
                return {"resultado": "contra-exemplo", "k": k, "traço": traco,
                        "lacete": None, "tempo": perf_counter() - inicio}
            solver.pop()
            # insatisfazível: inv vale no último estado de qualquer traço com k estados,
            # por isso pode ficar como restrição permanente (ajuda o solver nos k seguintes)
            solver.add_assertion(inv(estados[-1]))
    return {"resultado": "sem contra-exemplo até K", "k": K, "traço": None,
            "lacete": None, "tempo": perf_counter() - inicio}
```

In [ ]:
```python
def bmc_eventually(declare, init, trans, prop, K):
    """BMC genérico de F prop: procura um lacete onde prop nunca é verdade; devolve resultado, k, traço, início do lacete e tempo."""
    inicio = perf_counter()
    with Solver(name=SOLVER) as solver:
        estados = [declare(0)]
        solver.add_assertion(init(estados[0]))
        for k in range(1, K + 1):
            if k > 1:
                estados.append(declare(k - 1))
                solver.add_assertion(trans(estados[-2], estados[-1]))
            solver.add_assertion(Not(prop(estados[-1])))  # permanente: vale em todo o lacete
```

```
            extra = declare(k)  # estado extra que fecha o lacete
            solver.push()
            solver.add_assertion(trans(estados[-1], extra))
            solver.add_assertion(Or([
                And([EqualsOrIff(extra[c], anterior[c]) for c in extra])
for anterior in estados
            ]))
            if solver.solve():
                completo = extrair_traco(solver.get_model(), estados +
[extra])
                traco, fecho = completo[:-1], completo[-1]
                lacete = next((l for l, estado in enumerate(traco) if
estado == fecho), None)
                if lacete is None:  # não deve acontecer: o solver garantiu
que algum estado coincide
                    raise RuntimeError(f"k = {k}: o estado extra não
coincide com nenhum estado do traço")
                return {"resultado": "contra-exemplo", "k": k, "traço":
traco,
                        "lacete": lacete, "tempo": perf_counter() - inicio}
            solver.pop()  # o estado extra e o fecho do lacete só valiam
para este k
    return {"resultado": "sem contra-exemplo até K", "k": K, "traço": None,
            "lacete": None, "tempo": perf_counter() - inicio}
```

7. Propriedade 1 — nunca se alcança o estado de erro

Formulação. $G\,\neg\mathit{erro}$: em todos os estados alcançáveis há pelo menos uma célula normal viva.

Técnica. `bmc_always` com `inv = lambda s: Not(erro(s))`, para cada modo de transição.

Interpretação.

- *Contra-exemplo:* um traço mínimo desde o estado inicial até a grelha morrer toda. A propriedade é **falsa**, e o traço é confirmado com `traco_e_valido` e mostrado com `apresentar_grelha`.
- *Sem contra-exemplo até `K_MAX`:* a propriedade vale nos primeiros `K_MAX` estados, mas isto **não é uma prova** para execuções mais longas. Nesse caso o resultado é inconclusivo e indicamos o limite atingido.

In [ ]:
```
def simular_ate_ciclo(fronteira, n=N, passos_max=10_000):
    """Simula o modo síncrono a partir do estado inicial até um estado se
repetir.
    Devolve (traço sem repetições, índice onde o ciclo começa, período do
ciclo)."""
    grelha = estado_inicial_concreto(n)
    traco = []
    vistos = {}  # frozenset das células vivas -> índice onde o estado
```

```
apareceu
    while True:
        chave = frozenset(c for c, v in grelha.items() if v == 1)
        if chave in vistos:
            inicio = vistos[chave]
            return traco, inicio, len(traco) - inicio
        if len(traco) >= passos_max:
            raise RuntimeError(f"nenhum estado se repetiu em {passos_max}
passos")
        vistos[chave] = len(traco)
        traco.append(grelha)
        grelha = passo_concreto(grelha, fronteira, "sincrona", n=n)
```

In [ ]:
```
def verificar_nunca_erro(fronteira, modo=MODO_TRANSICAO, k_max=K_MAX, n=N):
    """Aplica bmc_always a G ¬erro com a FSM de fabricar_fsm; devolve o
dicionário de resultado do BMC."""
    declare, init_fsm, trans_fsm = fabricar_fsm(fronteira, modo, n)
    resultado = bmc_always(declare, init_fsm, trans_fsm, lambda s:
Not(erro(s, n)), k_max)
    if resultado["traço"] is not None:  # contra-exemplo: conferido pelo
simulador
        traco = resultado["traço"]
        problemas = traco_e_valido(traco, fronteira, modo, n)
        if sum(int(v) for v in traco[-1].values()) != 0:
            problemas.append("o último estado não é o estado de erro")
        anteriores = [k for k, g in enumerate(traco[:-1]) if sum(int(v) for
v in g.values()) == 0]
        if anteriores:
            problemas.append(f"contra-exemplo não mínimo: estados de erro
antes do fim em {anteriores}")
        resultado["validação"] = problemas
    return resultado
```

8. Propriedade 2 — nenhuma célula normal permanentemente morta ou viva

Formulação. Para cada célula normal c:

- não está permanentemente morta: $F\,(c = 1)$;
- não está permanentemente viva: $F\,(c = 0)$.

A propriedade 2 é a conjunção destas $2N^2$ fórmulas.

Técnica. `bmc_eventually` para cada célula e cada valor. Um contra-exemplo é um lacete em que a célula nunca muda, ou seja, uma execução infinita em que ela fica para sempre morta (ou viva). Basta uma célula com contra-exemplo para a propriedade 2 ser **falsa**.

Agregação. `verificar_todas_celulas` junta os resultados numa tabela (célula, valor, resultado, k, tempo). No modo síncrono, com a moldura fixa, o sistema é determinista: há uma única execução, que a simulação concreta reproduz. Por isso ela

mostra logo que células mudam e quais ficam constantes, o que serve para conferir os resultados do BMC e para escolher que células verificar primeiro. No modo assíncrono há muitas execuções possíveis, e uma simulação não chega para excluir um contra-exemplo. Aí só o BMC responde.

In [ ]:
```
def verificar_permanencia(celula, valor, fronteira, modo=MODO_TRANSICAO,
k_max=K_MAX, n=N):
    """Aplica bmc_eventually a F(célula ≠ valor) (0 = morta, 1 = viva);
devolve o dicionário de resultado do BMC."""
    if celula not in set(celulas_normais(n)):
        raise ValueError(f"{celula} não é uma célula normal para n = {n}")
    if valor not in (0, 1):
        raise ValueError(f"valor tem de ser 0 ou 1, recebido {valor!r}")

    # atalho: se a célula já é diferente de valor no estado inicial,
F(célula ≠ valor) vale no estado 0
    if estado_inicial_concreto(n)[celula] != valor:
        return {"resultado": "provada no estado inicial", "k": 1, "traço":
None,
                "lacete": None, "tempo": 0.0, "validação": None}

    declare, init_fsm, trans_fsm = fabricar_fsm(fronteira, modo, n)

    def prop(s):  # «a célula é diferente de valor»
        return s[celula] if valor == 0 else Not(s[celula])

    resultado = bmc_eventually(declare, init_fsm, trans_fsm, prop, k_max)
    resultado["validação"] = None
    if resultado["traço"] is not None:  # contra-exemplo: conferido pelo
simulador
        traco = resultado["traço"]
        problemas = traco_e_valido(traco + [traco[resultado["lacete"]]],
fronteira, modo, n)
        mudou = [k for k, g in enumerate(traco) if int(g[celula]) != valor]
        if mudou:
            problemas.append(f"a célula {celula} não vale {valor} nos
estados {mudou}")
        resultado["validação"] = problemas
    return resultado
```

In [ ]:
```
def verificar_todas_celulas(fronteira, modo=MODO_TRANSICAO, k_max=K_MAX,
n=N,
                            celulas=None, parar_no_primeiro=False):
    """Aplica verificar_permanencia a todas as células normais e devolve
uma tabela-resumo.
    A conclusão global fica em tabela.attrs["conclusão"]: falsa, provada ou
inconclusiva até K."""
    celulas = celulas_normais(n) if celulas is None else celulas
    linhas = []
    for c in celulas:
        for valor in (0, 1):
            r = verificar_permanencia(c, valor, fronteira, modo, k_max, n)
            linhas.append({
                "célula": c,
```

```
                "valor testado": "morta" if valor == 0 else "viva",
                "resultado": r["resultado"],
                "k": r["k"],
                "lacete": r["lacete"],
                "tempo (s)": round(r["tempo"], 2),
                "validação": "—" if r["validação"] is None else ("ok" if
not r["validação"] else "falhou"),
            })
            if parar_no_primeiro and r["resultado"] == "contra-exemplo":
                break
        if parar_no_primeiro and linhas[-1]["resultado"] == "contra-
exemplo":
            break

    tabela = pd.DataFrame(linhas)
    resultados = tabela["resultado"]
    if (resultados == "contra-exemplo").any():
        tabela.attrs["conclusão"] = "falsa"
    elif (resultados == "provada no estado inicial").all():
        tabela.attrs["conclusão"] = "provada"
    else:
        tabela.attrs["conclusão"] = "inconclusiva até K"
    return tabela
```

9. Resultados

Execução das duas propriedades nos dois modos de transição, com a mesma moldura (`SEMENTE`). Cada contra-exemplo é confirmado pelo simulador antes de ser apresentado.

In [ ]:
```
# Profundidades da secção 9 (ajustar depois de medir os tempos de uma
célula)
K_ERRO_ASSINCRONA = 15        # >= 10: o erro precisa de pelo menos 9
transições
K_PERMANENCIA_ASSINCRONA = 10
AMOSTRA_P2_ASSINCRONA = None  # None = todas as células; um número = só as
N mais afastadas
PARAR_P2_ASSINCRONA = True    # parar no primeiro contra-exemplo da
propriedade 2
LIMITE_K_SINCRONA = 60        # teto do K das verificações cruzadas no modo
síncrono
```

In [ ]:
```
def estado_validacao(resultado):
    """Resume a chave «validação» de um resultado: «—» sem contra-exemplo,
«ok» ou «falhou»."""
    validacao = resultado.get("validação")
    if validacao is None:
        return "—"
    return "ok" if not validacao else "falhou"
```

In [ ]:
```
moldura = construir_fronteira(N, SEMENTE)
traco_sinc, inicio_ciclo_sinc, periodo_sinc = simular_ate_ciclo(moldura, N)
```

```
# propriedade 1: primeiro estado do traço com todas as células mortas (None se não existir)
erro_sinc = next((k for k, g in enumerate(traco_sinc) if sum(g.values()) == 0), None)

# propriedade 2: valores de cada célula no traço inteiro (leitura a) e só no ciclo (leitura b)
categoria_sinc = {}
constante_no_ciclo_sinc = []
for _c in celulas_normais(N):
    _no_traco = {g[_c] for g in traco_sinc}
    _no_ciclo = {g[_c] for g in traco_sinc[inicio_ciclo_sinc:]}
    if _no_traco == {0}:
        categoria_sinc[_c] = "sempre morta"
    elif _no_traco == {1}:
        categoria_sinc[_c] = "sempre viva"
    else:
        categoria_sinc[_c] = "muda"
    if len(_no_ciclo) == 1:
        constante_no_ciclo_sinc.append(_c)
```

In [ ]:

```
_contagem = {cat: sum(1 for v in categoria_sinc.values() if v == cat)
             for cat in ("sempre morta", "sempre viva", "muda")}
_constantes = _contagem["sempre morta"] + _contagem["sempre viva"]
_simbolo = {"sempre morta": "·", "sempre viva": "■", "muda": "◆"}
_mapa = pd.DataFrame(
    [[_simbolo[categoria_sinc[(i, j)]] for j in range(1, N + 1)] for i in range(1, N + 1)],
    index=range(1, N + 1), columns=range(1, N + 1),
)

if erro_sinc is None:
    _p1 = (f"**provada.** Nenhum dos {len(traco_sinc)} estados alcançáveis é o estado de erro. "
           "Como o modo síncrono é determinista e tem um único estado inicial, estes são "
           "todos os estados alcançáveis.")
else:
    _p1 = (f"**falsa.** O estado de erro é alcançado no estado {erro_sinc} "
           f"(traço de {erro_sinc + 1} estados).")

mo.vstack([
    mo.md(f"""
### Moldura e simulação síncrona

Moldura sorteada com `SEMENTE = {SEMENTE}`, N = {N}. Estado inicial com a moldura
(■ viva, · morta nas células normais; ● viva, ○ morta na moldura):
"""),
    mo.Html(apresentar_grelha(estado_inicial_concreto(N), moldura, N).to_html()),
    mo.md(f"""
A execução síncrona tem **{len(traco_sinc)} estados distintos**: transiente de
{inicio_ciclo_sinc} estados e ciclo de período {periodo_sinc}
```

```
{"(ponto fixo)" if periodo_sinc == 1 else "(oscilador)"}.

**Propriedade 1 (síncrono):** {_p1}

**Propriedade 2 (síncrono), leitura (a):** {_contagem["sempre morta"]}
células sempre
mortas, {_contagem["sempre viva"]} sempre vivas e {_contagem["muda"]} que
mudam.
A propriedade é **{"falsa" if _constantes else "provada"}**.

**Leitura (b):** {len(constante_no_ciclo_sinc)} células ficam constantes
dentro do ciclo.

Mapa das células (· sempre morta, ■ sempre viva, ◆ muda):
"""),
    mo.Html(_mapa.to_html()),
])
```

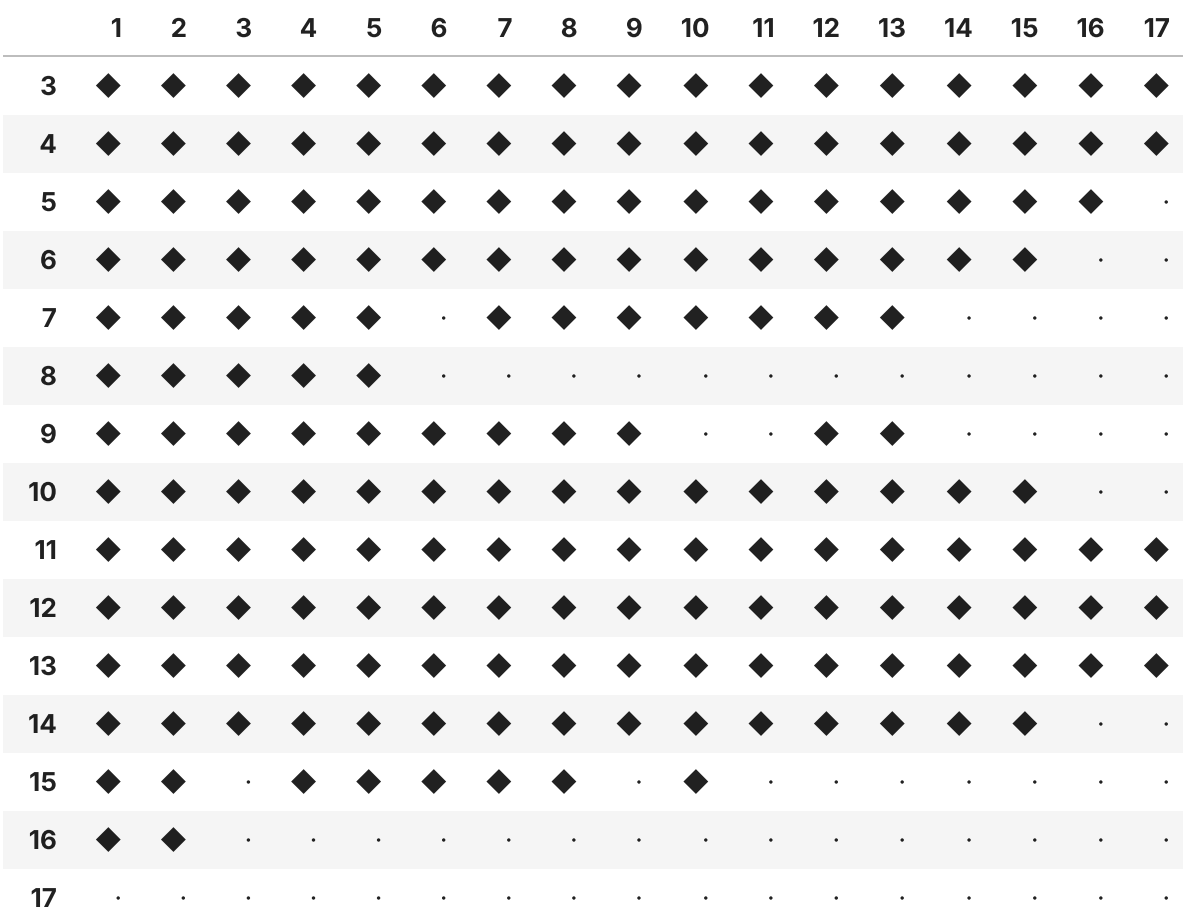
Moldura e simulação síncrona

Moldura sorteada com `SEMENTE = 0`, N = 17. Estado inicial com a moldura (■ viva, · morta nas células normais; ● viva, ○ morta na moldura):

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0	○	○	●	●	○	●	○	●	●	○	○	○	●	○	○	●	○	○	○
1	○	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
2	○	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
3	●	■	■	■	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
4	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
5	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
6	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
7	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
8	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
9	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
10	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
11	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
12	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
13	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
14	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
15	●	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
16	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
17	○	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	○
18	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○

A execução síncrona tem **157 estados distintos**: transiente de 156 estados e ciclo de período 1 (ponto fixo). **Propriedade 1 (síncrono): provada.** Nenhum dos 157 estados alcançáveis é o estado de erro. Como o modo síncrono é determinista e tem um único estado inicial, estes são todos os estados alcançáveis. **Propriedade 2 (síncrono), leitura (a):** 71 células sempre mortas, 0 sempre vivas e 218 que mudam. A propriedade é **falsa**. **Leitura (b):** 289 células ficam constantes dentro do ciclo. Mapa das células (· sempre morta, ■ sempre viva, ◆ muda):

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
2	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
3	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
4	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
5	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	·
6	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	·	·
7	◆	◆	◆	◆	◆	·	◆	◆	◆	◆	◆	◆	◆	·	·	·	·
8	◆	◆	◆	◆	◆	·	·	·	·	·	·	·	·	·	·	·	·
9	◆	◆	◆	◆	◆	◆	◆	◆	◆	·	·	◆	◆	·	·	·	·
10	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	·	·
11	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
12	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
13	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆
14	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	◆	·	·
15	◆	◆	·	◆	◆	◆	◆	◆	·	◆	·	·	·	·	·	·	·
16	◆	◆	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
17	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·

In [ ]:
```
botao_p1 = mo.ui.run_button(label="Propriedade 1 — BMC")
botao_p2_sinc = mo.ui.run_button(label="Propriedade 2 — verificação cruzada
(síncrono)")
botao_p2_assinc = mo.ui.run_button(label="Propriedade 2 — BMC
(assíncrono)")
mo.hstack([botao_p1, botao_p2_sinc, botao_p2_assinc], justify="start")
```

Propriedade 1 — BMC | Propriedade 2 — verificação cruzada (síncrono) | Propriedade 2 — BMC (assíncrono)

In [ ]:
```
mo.stop(not botao_p1.value, mo.md("_Carrega em «Propriedade 1 — BMC» para
correr esta verificação._"))

k_p1_sinc = min(len(traco_sinc), LIMITE_K_SINCRONA)
p1_sinc = verificar_nunca_erro(moldura, "sincrona", k_p1_sinc, N)
p1_assinc = verificar_nunca_erro(moldura, "assincrona", K_ERRO_ASSINCRONA,
N)

# verificação cruzada no síncrono: o BMC tem de concordar com a simulação
if erro_sinc is not None and erro_sinc + 1 <= k_p1_sinc:
    _traco_bmc = [{c: int(v) for c, v in g.items()} for g in
(p1_sinc["traço"] or [])]
    p1_concorda = p1_sinc["k"] == erro_sinc + 1 and _traco_bmc ==
traco_sinc[: erro_sinc + 1]
else:  # sem erro ao alcance de K: o BMC não pode encontrar contra-exemplo
    p1_concorda = p1_sinc["resultado"] != "contra-exemplo"

mo.md(f"""
```

```
### Propriedade 1 — BMC

**Síncrono** (K = {k_p1_sinc}): {p1_sinc["resultado"]}, k = {p1_sinc["k"]},
{p1_sinc["tempo"]:.2f} s, validação: {estado_validacao(p1_sinc)}.
Concorda com a simulação: {"sim ✅" if p1_concorda else "não ❌"}.

**Assíncrono** (K = {K_ERRO_ASSINCRONA}): {p1_assinc["resultado"]}, k =
{p1_assinc["k"]},
{p1_assinc["tempo"]:.2f} s, validação: {estado_validacao(p1_assinc)}.
""")
```

Carrega em «Propriedade 1 — BMC» para correr esta verificação.

In [ ]:

```
mo.stop(not botao_p2_sinc.value,
        mo.md("_Carrega em «Propriedade 2 — verificação cruzada
(síncrono)»._"))

_L = len(traco_sinc)
_mortas = [c for c, cat in categoria_sinc.items() if cat == "sempre morta"]
_vivas = [c for c, cat in categoria_sinc.items() if cat == "sempre viva"]
# as duas últimas sempre mortas (as mais afastadas do quadrado inicial) e
uma sempre viva
_amostra = [(c, 0) for c in _mortas[-2:]] + [(c, 1) for c in _vivas[:1]]

_linhas = []
if _amostra and _L <= LIMITE_K_SINCRONA:
    for _c, _valor in _amostra:
        _r = verificar_permanencia(_c, _valor, moldura, "sincrona", _L, N)
        _linhas.append({
            "célula": str(_c), "valor testado": "morta" if _valor == 0 else
"viva", "K": _L,
            "resultado": _r["resultado"], "k": _r["k"], "k esperado": _L,
            "lacete": _r["lacete"], "lacete esperado": inicio_ciclo_sinc,
            "validação": estado_validacao(_r), "tempo (s)":
round(_r["tempo"], 2),
            "concorda": _r["k"] == _L and _r["lacete"] == inicio_ciclo_sinc
                        and estado_validacao(_r) == "ok",
        })
    if _L > 1:  # limitação do BMC: com K abaixo do comprimento do traço, o
lacete não fecha
        _c, _valor = _amostra[0]
        _r = verificar_permanencia(_c, _valor, moldura, "sincrona", _L - 1,
N)
        _linhas.append({
            "célula": str(_c), "valor testado": "morta" if _valor == 0 else
"viva", "K": _L - 1,
            "resultado": _r["resultado"], "k": _r["k"], "k esperado": "—",
            "lacete": _r["lacete"], "lacete esperado": "—",
            "validação": estado_validacao(_r), "tempo (s)":
round(_r["tempo"], 2),
            "concorda": _r["resultado"] != "contra-exemplo",
        })
tabela_p2_sinc = pd.DataFrame(_linhas)

if not _amostra:
    _saida = mo.md("Não há células constantes no modo síncrono: não há
```

```
contra-exemplos para confirmar.")
elif _L > LIMITE_K_SINCRONA:
    _saida = mo.md(f"O traço síncrono tem {_L} estados, acima de
`LIMITE_K_SINCRONA` = "
                   f"{LIMITE_K_SINCRONA}: a verificação cruzada com BMC não
foi feita.")
else:
    _saida = mo.vstack([
        mo.md(f"""
### Propriedade 2 — verificação cruzada (síncrono)

Para as células constantes, o BMC com K = {_L} tem de encontrar um lacete
com k = {_L}
que começa no estado {inicio_ciclo_sinc}. A última linha usa K = {_L - 1}:
aí o lacete
ainda não fecha e o BMC não encontra o contra-exemplo, embora ele exista.
"""),
        tabela_p2_sinc,
    ])
_saida
```

Carrega em «Propriedade 2 — verificação cruzada (síncrono)».

In [ ]:

```
mo.stop(not botao_p2_assinc.value, mo.md("_Carrega em «Propriedade 2 — BMC
(assíncrono)»._"))

# células mais afastadas do quadrado inicial primeiro: são as que mais
provavelmente ficam constantes
_ordem = sorted(celulas_normais(N), key=lambda c: max(c[0] - 3, c[1] - 3,
0), reverse=True)
tabela_p2_assinc = verificar_todas_celulas(
    moldura, "assincrona", K_PERMANENCIA_ASSINCRONA, N,
    celulas=_ordem[:AMOSTRA_P2_ASSINCRONA],
parar_no_primeiro=PARAR_P2_ASSINCRONA,
)
_falhou = (tabela_p2_assinc["validação"] == "falhou").any()
conclusao_p2_assinc = "erro de validação" if _falhou else
tabela_p2_assinc.attrs["conclusão"]

# repetir o primeiro contra-exemplo para ter o traço (o BMC é determinista:
dá o mesmo)
_cex = tabela_p2_assinc[tabela_p2_assinc["resultado"] == "contra-exemplo"]
if len(_cex):
    _celula = _cex.iloc[0]["célula"]
    _valor = 0 if _cex.iloc[0]["valor testado"] == "morta" else 1
    p2_assinc_exemplo = (_celula, _valor, verificar_permanencia(
        _celula, _valor, moldura, "assincrona", K_PERMANENCIA_ASSINCRONA,
N))
else:
    p2_assinc_exemplo = None

_mostrar = tabela_p2_assinc.copy()
_mostrar["célula"] = _mostrar["célula"].astype(str)
mo.vstack([
    mo.md(f"""
### Propriedade 2 — BMC (assíncrono)
```

```
K = {K_PERMANENCIA_ASSINCRONA}; {len(tabela_p2_assinc)} verificações em
{tabela_p2_assinc["tempo (s)"].sum():.1f} s. Conclusão: **
{conclusao_p2_assinc}**.
"""),
    _mostrar,
])
```

Carrega em «Propriedade 2 — BMC (assíncrono)».

In [ ]:

```
def _texto_bmc(r):
    if r["resultado"] == "contra-exemplo":
        return "falsa" if estado_validacao(r) == "ok" else "erro de
validação"
    return f"inconclusiva até K = {r['k']}"

_constantes = sum(1 for v in categoria_sinc.values() if v != "muda")
_val_p2_sinc = ("—" if tabela_p2_sinc.empty
                else "ok" if (tabela_p2_sinc["validação"] !=
"falhou").all() else "falhou")
_cex_p2 = tabela_p2_assinc[tabela_p2_assinc["resultado"] == "contra-
exemplo"]
_resultado_p2_assinc = {
    "falsa": "falsa",
    "provada": "provada",
    "inconclusiva até K": f"inconclusiva até K =
{K_PERMANENCIA_ASSINCRONA}",
}.get(conclusao_p2_assinc, conclusao_p2_assinc)

resumo = pd.DataFrame([
    {"propriedade": "1 — nunca erro", "modo": "síncrono",
     "método": "simulação até ao ciclo + BMC (verificação cruzada)",
     "resultado": "provada" if erro_sinc is None else "falsa",
     "k / K": "—" if erro_sinc is None else str(erro_sinc + 1),
     "tempo (s)": round(p1_sinc["tempo"], 2),
     "validação": estado_validacao(p1_sinc) + ("" if p1_concorda else "
(BMC discorda!)")},
    {"propriedade": "1 — nunca erro", "modo": "assíncrono", "método":
"BMC",
     "resultado": _texto_bmc(p1_assinc), "k / K": str(p1_assinc["k"]),
     "tempo (s)": round(p1_assinc["tempo"], 2), "validação":
estado_validacao(p1_assinc)},
    {"propriedade": "2 — nenhuma célula permanente", "modo": "síncrono",
     "método": "simulação até ao ciclo + BMC (amostra)",
     "resultado": "falsa" if _constantes else "provada",
     "k / K": str(len(traco_sinc)) if _constantes else "—",
     "tempo (s)": round(tabela_p2_sinc["tempo (s)"].sum(), 2) if not
tabela_p2_sinc.empty else 0.0,
     "validação": _val_p2_sinc},
    {"propriedade": "2 — nenhuma célula permanente", "modo": "assíncrono",
     "método": "BMC" + ("" if AMOSTRA_P2_ASSINCRONA is None else f"
(amostra de {AMOSTRA_P2_ASSINCRONA})"),
     "resultado": _resultado_p2_assinc,
     "k / K": str(_cex_p2.iloc[0]["k"]) if len(_cex_p2) else
str(K_PERMANENCIA_ASSINCRONA),
     "tempo (s)": round(tabela_p2_assinc["tempo (s)"].sum(), 2),
```

```
    "validação": "ok" if conclusao_p2_assinc != "erro de validação" else
"falhou"},
])
mo.vstack([mo.md("### Resumo"), resumo])
```

In [ ]:
```
_ver = lambda g: mo.Html(apresentar_grelha(g, moldura, N).to_html())
_blocos = [mo.md("### Contra-exemplos")]

for _nome, _r in (("síncrono", p1_sinc), ("assíncrono", p1_assinc)):
    if _r["resultado"] == "contra-exemplo" and estado_validacao(_r) ==
"ok":
        _t = _r["traço"]
        _blocos += [
            mo.md(f"**Propriedade 1, modo {_nome}:** traço de {len(_t)}
estados. "
                  "À esquerda o estado inicial, à direita o estado de
erro."),
            mo.hstack([_ver(_t[0]), _ver(_t[-1])], justify="start"),
        ]

_constantes_sinc = [c for c, cat in categoria_sinc.items() if cat !=
"muda"]
if _constantes_sinc:
    _c = _constantes_sinc[-1]
    _blocos += [
        mo.md(f"**Propriedade 2, modo síncrono:** a célula {_c} está
{categoria_sinc[_c]}. "
              f"Cabo de {inicio_ciclo_sinc} estados e laço de
{periodo_sinc}. "
              "Estado onde o laço começa:"),
        _ver(traco_sinc[inicio_ciclo_sinc]),
    ]

if p2_assinc_exemplo is not None:
    _c, _valor, _r = p2_assinc_exemplo
    _blocos += [
        mo.md(f"**Propriedade 2, modo assíncrono:** a célula {_c} fica
sempre "
              f"{'morta' if _valor == 0 else 'viva'}. Traço de {_r['k']}
estados: cabo de "
              f"{_r['lacete']} e laço de {_r['k'] - _r['lacete']}. Estado
onde o laço começa:"),
        _ver(_r["traço"][_r["lacete"]]),
    ]

mo.vstack(_blocos) if len(_blocos) > 1 else mo.md("_Não foram encontrados
contra-exemplos._")
```

In [ ]:
```
_frases = "\n".join(
    f"- **Propriedade {linha['propriedade']}**, modo {linha['modo']}:
{linha['resultado']} "
    f"({linha['método']}; k / K = {linha['k / K']}; {linha['tempo (s)']}
s)."
    for _, linha in resumo.iterrows()
```

```
)
mo.md(f"### Resultados obtidos\n\n{_frases}")
```

10. Conclusão

Texto redigido com apoio de um LLM (Claude) a partir dos resultados da secção 9.

Resultados com $N = 17$ e a moldura sorteada com `SEMENTE = 0`:

- **Propriedade 1 (nunca se alcança o erro).**
  - *Síncrono:* **provada.** O modo é determinista e há um só estado inicial, por isso a simulação percorre todos os estados alcançáveis: 157 estados distintos, até a grelha parar num ponto fixo. Nenhum é o estado de erro, e o BMC até $K = 60$ concorda.
  - *Assíncrono:* **falsa.** O BMC encontra um contra-exemplo com 10 estados (9 transições) em que todas as células normais morrem. O traço foi confirmado pelo simulador.
- **Propriedade 2 (nenhuma célula fica para sempre morta ou viva). Falsa nos dois modos.**
  - *Síncrono:* 71 células nunca chegam a estar vivas.
  - *Assíncrono:* o BMC encontra, com $k = 6$, um lacete em que a célula $(1, 17)$ fica sempre morta. O traço também foi confirmado pelo simulador.

Comparação dos modos. No modo síncrono há uma única execução, que a simulação esgota, por isso os resultados são provas. No modo assíncrono o solver escolhe que célula muda em cada passo, o que abre muitas mais execuções. É isso que permite chegar ao estado de erro, que no modo síncrono nunca acontece.

Limites. O BMC só procura traços até K estados, por isso, sem contra-exemplo, o resultado é inconclusivo para lá de K. Aqui isso não pesa: o único resultado positivo (propriedade 1 no síncrono) vem da simulação completa, e os resultados negativos são contra-exemplos concretos, validados pelo simulador. Todos os resultados valem para esta moldura; outra semente pode dar resultados diferentes.

Os testes automáticos estão no notebook `test_automato_celular.py`.

Uso de ferramentas LLM

Diferenças entre a conversa e o código final

- **Ferramenta de verificação.** No Registo 2 o LLM recomendou o nuXmv e sugeriu não usar BMC. Não seguimos essa recomendação: usamos pySMT com Z3 e BMC, como na ficha 7, por serem as ferramentas e a técnica da cadeira.

- **Funções de verificação.** A estrutura do Registo 1 tinha `construir_sistema` e `verificar`, pensadas para um model checker externo. Foram substituídas por `fabricar_fsm`, que devolve `(declare, init, trans)`, e pelas funções genéricas `bmc_always` e `bmc_eventually`, com a mesma assinatura da ficha 7.
- **Funções acrescentadas.** `construir_fronteira` (valores fixos da moldura) e `extrair_traco` (leitura do traço a partir do modelo do solver), que não existiam na estrutura proposta.
- **Nomes dos ficheiros.** `game_of_life_fsm.py` passou a `automato_celular.py`, e o apêndice foi dividido por elemento do grupo.

Apêndice — registos de orientação do projeto

Este ficheiro guarda as transcrições de apoio usadas durante a elaboração. A documentação técnica e os resultados finais estão em `automato_celular.py`.

Declaração de uso de LLMs

Ferramenta usada: **Claude**. Os registos abaixo vêm da conversa de apoio ao trabalho.

Registo 1

Registo de apoio — estrutura do projeto

Prompt (Claude):

> (enunciado do Exercício 1 do Trabalho Prático 2: Game of Life como FSM, com as duas interpretações das transições e as duas propriedades)
>
> gera me uma estrutura para este projeto, tendo em conta este meu trabalho pratico desenvolvido no ambito da mesma cadeira, sem solucoes. é para marimo

Resposta do LLM:

(O LLM gerou o ficheiro `game_of_life_fsm.py` com a estrutura do notebook e as funções por implementar. Depois de ajustado, explicou:)

Tirei do `game_of_life_fsm.py` tudo o que era BMC feito à mão: o parâmetro `K_MAX`, a função `gerar_traco` (que desenrolava k estados) e as referências a profundidade e critério de paragem. O notebook continua a correr do início ao fim.

Agora a verificação divide-se assim:

- `construir_sistema` junta as variáveis de estado, o `init` e a `trans` no formato de sistema de transição que a ferramenta recebe.
- `verificar` entrega o sistema e a propriedade ao model checker da própria ferramenta e devolve o resultado, o contra-exemplo e o tempo.
- `verificar_nunca_erro` e `verificar_permanencia` só escrevem cada propriedade e chamam `verificar`.

Na secção 2 há um ponto novo, Verificação, para justificares que motor da ferramenta usas e o que ele garante. O simulador e o `traco_e_valido` ficaram, para confirmares de forma independente os contra-exemplos que o model checker devolver.

Que ferramenta vão usar? Assim que me disseres, ajusto as dependências e os nomes ao que ela espera.

Registo 2

Registo de apoio — escolha da ferramenta

Prompt (Claude):

> que ferramenta recomendas?

Resposta do LLM:

Recomendo o nuXmv. Escreves o modelo em linguagem SMV a partir do Python, no próprio notebook, e corres o nuXmv por `subprocess`.

Porque é que encaixa neste exercício:

- É um model checker completo. Defines a máquina de estados e a ferramenta faz a verificação toda, sem BMC feito à mão. Para sistemas finitos tem algoritmos IC3, interpolação e K-liveness, que dão provas completas e não apenas até uma profundidade.
- As duas relações de transição escrevem-se diretamente em SMV. A síncrona e a assíncrona são ambas naturais, e o parâmetro `MODO_TRANSICAO` só decide qual delas é gerada.
- Verifica tanto invariantes como propriedades temporais. A propriedade 2 («nenhuma célula permanentemente morta ou viva») não é um simples invariante. Uma ferramenta que só prova invariantes obrigar-te-ia a codificações extra para essa propriedade.
- Devolve contra-exemplos como traços. Podes confirmá-los com o simulador do esqueleto (`traco_e_valido`).

Cuidados a ter:

- Licença. O nuXmv não é open source, mas é gratuito para uso académico e não comercial.
- Instalação. Tens de descarregar o binário à parte, porque não vem pelo pip. Isso tem de ficar escrito no README, já que o `uvx marimo --sandbox` não o instala.
- PyNuSMV. É a ligação Python ao NuSMV, mas está parada: a última versão é de 2017, para Python 3.5/3.6. Por isso recomendo gerar o `.smv` e chamar o binário.

- Tamanho do estado. Com N = 17 tens 289 variáveis de estado. Os motores baseados em BDD podem não aguentar, e é por isso que deves usar os motores IC3/SAT.

Alternativas:

- Z3 com o motor Spacer. Instala-se pelo pip (`z3-solver`) e prova invariantes de forma completa. Mas só trata propriedades de segurança, por isso a propriedade 2 dava muito mais trabalho.
- pySMT. Não tem model checker incorporado, logo obrigava-te precisamente a implementar o BMC à mão.

Antes de decidires, confirma com o docente se ferramentas externas à cadeira são aceites, porque no TP1.1 havia uma sugestão oficial (o CP-SAT). Se escolheres o nuXmv, adapto o esqueleto: as dependências, o README com a instalação e as funções `construir_sistema` e `verificar` com a assinatura de gerar o `.smv` e ler o resultado, sem as implementar.

Fontes:

- nuXmv — licença
- nuXmv — funcionalidades
- nuXmv — download
- PyNuSMV no PyPI

Registo 3

Registo de apoio — guia para `celulas_normais` e `celulas_iniciais`

Prompt (Claude):

> guia me agora no desenvolvimento das duas primeiras funcoe, sem solucoes

Resposta do LLM:

Definindo o formato das saídas e cuidados.

As duas primeiras funções são `celulas_normais` e `celulas_iniciais`. São simples, mas tudo o resto (vizinhos, estados, transições, propriedades) é construído a partir delas, por isso vale a pena acertar já três decisões.

Decisões comuns às duas

- Representação de cada célula. O esqueleto usa o par `(i, j)`. Mantém essa convenção em todo o notebook, porque os estados vão ser dicionários indexados por estes pares.
- Lista ou conjunto. Um conjunto facilita testes de pertença. Uma lista tem ordem fixa, e a ordem importa. Quando, mais à frente, `sortear_iniciais` percorrer as células e sortear um valor para cada uma, a ordem de percurso decide que valor calha a que célula. Com uma ordem fixa, a mesma `SEMENTE` dá sempre a mesma fronteira, o que precisas para resultados reproduzíveis. Escolhe e justifica numa frase na secção 3.
- Parâmetro `n`. Usa sempre o `n` recebido e nunca a constante `N` dentro da função. É isso que permite o teste com outro tamanho de grelha.

`celulas_normais(n)`

O que deve devolver: todas as células com os dois índices entre 1 e `n`, inclusive.

Pontos de atenção:

- Os índices do enunciado começam em 1, não em 0. Confirma os limites dos `range` do Python, que excluem o valor final.
- No `app.setup` tens `itertools.product` importado. Vê se te serve aqui.

Como verificar:

- Com `n = 3`, deves obter 9 células, sem nenhuma com índice 0 nem 4.
- Para um `n` qualquer, o total tem de ser `n²`.

`celulas_iniciais(n)`

O que deve devolver: as células em que pelo menos um índice é 0.

Antes de programar, falta decidir até onde vai o outro índice. O enunciado não o diz diretamente, mas dá duas contagens que, juntas, o determinam: $2N+1$ células iniciais e $2N+3$ finais.

Sugiro este exercício:

1. Desenha a moldura completa para `n = 2`, com os índices de 0 a 3 nos dois eixos.
2. Marca as células normais.
3. Na moldura sobram três cantos ambíguos: `(0, 0)`, `(0, n+1)` e `(n+1, 0)`. Decide a que conjunto pertence cada um, de forma a que as contagens de iniciais e de finais batam certo ao mesmo tempo.
4. Confirma que nenhuma célula da moldura fica sem conjunto.

Armadilha: se construíres o resultado juntando a «linha» com `i = 0` e a «coluna» com `j = 0`, há uma célula que aparece nas duas. Comparar o tamanho da lista com o tamanho de `set(lista)` deteta isso.

Como verificar:

- A contagem tem de ser `2n + 1` para vários valores de `n`.
- Não pode haver repetidos.
- Nenhuma célula inicial pode ser também normal.

Esta última verificação e a da contagem correspondem ao `test_numero_de_celulas` e ao `test_conjuntos_disjuntos` do notebook de testes. Quando as duas funções estiverem feitas, já podes trocar o `skipTest` dessas partes pelas verificações.

Como no TP1.1, acrescenta a cada função o comentário `# LLM (Claude) — Registo N: ...` e copia esta conversa para o `apendice_conversas.md`.

Quando tiveres as duas funções feitas, envia-mas e eu reviso antes de passarmos a `celulas_finais` e `vizinhos`.

Registo 4

Registo de apoio — guia para `celulas_finais` e `vizinhos`

Prompt (Claude):

> otimo. vamos prosseguir com as proximas duas funcoes

Resposta do LLM:

Checando a união dos três conjuntos de células

As duas funções seguintes são `celulas_finais` e `vizinhos`. A primeira fecha a moldura da grelha; a segunda define quem conta para a regra B3/S23 de cada célula.

`celulas_finais(n)`

O que deve devolver: as células com um índice igual a `n+1` que não sejam iniciais.

O «não sejam iniciais» do enunciado é a pista principal. A forma mais direta de cumprir essa condição é reaproveitar `celulas_iniciais`, em vez de repetir à mão a decisão sobre os cantos. Assim, se um dia mudares a decisão numa função, a outra acompanha.

Pontos de atenção:

- Cantos. A decisão que tomaste sobre `(0, n+1)`, `(n+1, 0)` e `(n+1, n+1)` tem de dar exatamente $2n+3$ células finais.
- Repetidos. A mesma armadilha da função anterior: a linha `i = n+1` e a coluna `j = n+1` partilham uma célula.

- Ordem. Usa a mesma convenção de lista ou conjunto que escolheste para as outras duas funções.

Como verificar: as três funções juntas têm de cobrir a moldura inteira, de 0 a `n+1` nos dois eixos, sem sobreposições. As contagens batem certo, porque $n^2+(2n+1)+(2n+3)=(n+2)^2$. Por isso, confirma:

- que a união dos três conjuntos é exatamente o quadrado de `(n+2)×(n+2)` posições;
- que nenhum par de conjuntos se interseta.

Esta é a melhor prova de que os cantos ficaram bem decididos, e é praticamente o `test_conjuntos_disjuntos`.

`vizinhos(i, j, n)`

O que deve devolver: as células à distância 1 de `(i, j)` segundo o enunciado, $\max(|i_0-i_1|, |j_0-j_1|)=1$.

Antes de programar, decide duas coisas:

1. A célula conta como vizinha de si própria? Aplica a fórmula a `(i, j)` com ela mesma e vê o que dá.

2. Para que células é chamada a função? Só as células normais mudam de estado, por isso só elas precisam dos vizinhos. Repara que, se `(i, j)` for normal, todos os vizinhos caem dentro da moldura de 0 a `n+1`: nunca saem da grelha. Se a função for chamada para uma célula da moldura, isso já não é verdade. Escolhe entre:

   - aceitar só células normais e lançar `ValueError` para as outras;
   - aceitar qualquer célula e filtrar as posições que não existem.

   Escolhe uma e justifica-a numa frase.

Implementação: há duas formas razoáveis.

- Gerar os deslocamentos possíveis à volta de `(i, j)`.
- Percorrer as posições candidatas e ficar com as que cumprem a condição do enunciado.

A segunda segue a definição à letra, o que dá jeito para a justificação no relatório. A primeira é mais curta. Qualquer uma serve.

Eficiência (opcional): `vizinhos` vai ser chamada para cada célula sempre que construíres uma transição. Com N = 17 não faz diferença. Se mais à frente quiseres testar grelhas maiores, podes calcular os vizinhos uma vez e guardá-los num dicionário.

Como verificar: é o `test_vizinhanca`.

- Célula interior, por exemplo `(5, 5)`: deve ter o número de vizinhos que esperas, todos normais.
- Canto `(1, 1)`: deve ter o mesmo número de vizinhos. Alguns serão iniciais; confirma quais.
- Canto `(1, n)`: é o caso mais revelador, porque os vizinhos misturam células normais, iniciais e finais. Se a decisão sobre `(0, n+1)` estiver errada, é aqui que se nota.
- Canto `(n, n)`: os vizinhos fora da grelha normal devem ser todos finais.

Uma nota para mais à frente: `vizinhos` devolve só posições. Saber se cada vizinha está viva (variável do solver nas normais, constante nas iniciais e finais) é trabalho de `valor_celula`. Mantém essa separação: assim, `vizinhos` serve tanto para o modelo pySMT como para o simulador concreto.

Quando tiveres estas duas funções, podes trocar o `skipTest` de `test_numero_de_celulas`, `test_conjuntos_disjuntos` e `test_vizinhanca` pelas verificações. A seguir vem `sortear_iniciais`, que fecha a secção 3.

Registo 5

Registo de apoio — testes da geometria da grelha

Prompt (Claude):

> gera me agora os testes, so para confirmar que o codigo esta correto

Resposta do LLM:

Vou preencher no notebook de testes as verificações das quatro funções que já fizeste: `celulas_normais`, `celulas_iniciais`, `celulas_finais` e `vizinhos`. Os testes das funções que ainda faltam ficam ignorados.

(O LLM criou o ficheiro de testes e executou-o.)

Agora confirmo que os testes estão certos: corro-os contra uma implementação de referência minha (que fica só no rascunho, não a envio) e contra versões com erros típicos, que têm de falhar.

(O LLM partilhou o ficheiro `test_game_of_life_fsm.py`.)

Os testes das quatro funções já feitas (`celulas_normais`, `celulas_iniciais`, `celulas_finais` e `vizinhos`) estão no `test_game_of_life_fsm.py`. Para saber se o teu código está certo, coloca este ficheiro na pasta do projeto, ao lado do teu `game_of_life_fsm.py`, e abre-o no Marimo.

Testei-os contra uma versão correta minha, que não envio, e passaram todos. Testei-os também contra seis erros típicos, e cada um fez falhar pelo menos um teste:

- um canto da moldura no conjunto errado;
- uma célula inicial repetida;
- índices a começar em 0;
- a célula contada como vizinha de si própria;
- só 4 vizinhos;
- usar `N` em vez de `n`.

O que verificam (9 testes):

- Contagens: n^2 normais, $2n+1$ iniciais e $2n+3$ finais, sem repetidos. Correm com `n` = 2, 3, 5, 17 e 20, o que apanha o uso de `N` em vez de `n`. Por isso tirei o antigo `test_outro_tamanho_de_grelha`.
- Índices e conjuntos: os índices de cada conjunto estão certos, os três conjuntos não se sobrepõem e, juntos, cobrem a moldura toda.
- Cantos da moldura: cada canto está no conjunto que as contagens obrigam.
- Vizinhos: quantos vizinhos normais, iniciais e finais tem uma célula interior e cada um dos quatro cantos. Para todas as células normais confirma ainda os 8 vizinhos pela definição do enunciado e que a vizinhança é simétrica.

Os testes aceitam que as funções devolvam listas, conjuntos ou geradores. A `vizinhos` só é chamada para células normais, por isso tanto serve a opção de lançar `ValueError` como a de filtrar as posições que não existem.

Os 10 testes das funções que ainda faltam continuam marcados como ignorados.

Como no TP1.1, o notebook indica que os testes foram gerados pelo Claude e transcreve o teu pedido. O código gerado está entre os comentários `INÍCIO` e `FIM DO CÓDIGO GERADO POR LLM`.

Se algum teste falhar, envia-me a mensagem de erro.

Link da conversa: (https://claude.ai/share/15f59df4-05b5-4b76-ad81-621017166ab4)

Apêndice — registos de orientação do projeto (Nelson)

Este ficheiro guarda as transcrições de apoio usadas durante a elaboração. A documentação técnica e os resultados finais estão em `automato_celular.py`.

Declaração de uso de LLMs

Ferramenta usada: **Claude**. Os registos abaixo vêm da conversa de apoio ao trabalho. A numeração continua a do apêndice do André (Registos 1 a 4).

Registo 5

Registo de apoio — objetivo do projeto e ponto de situação

Prompt (Claude):

> Analisa este enunciado e o código anexado, explica o que é pretendido neste projeto, o ponto de situação em que se encontra e o que devo fazer em seguida
>
> (enunciado do Exercício 1 do Trabalho Prático 2, enviado com `automato_celular.py`, `test_automato_celular.py` e os apêndices)

Resposta do LLM:

(O LLM resumiu o objetivo do projeto e o estado do código:)

- **Objetivo:** modelar uma variante finita do Game of Life como FSM, com estado formado pelas N^2 células normais, moldura constante e duas relações de transição (síncrona e assíncrona) escolhidas por `MODO_TRANSICAO`. Depois, provar ou refutar $G\,\neg\mathit{erro}$ e a não permanência de cada célula, com BMC em pySMT/Z3, como na ficha 7.
- **Feito:** estrutura do notebook, texto das secções 1 a 8 e as funções `celulas_normais`, `celulas_iniciais` e `celulas_finais`, corretas nas contagens e nos cantos.
- **Em falta:** a função `vizinhos`, todas as restantes funções, os comentários `# LLM (Claude) — Registo N`, este apêndice e os testes.
- **Ordem sugerida:** `vizinhos` e a fronteira, depois o simulador, o modelo simbólico, o BMC genérico (testado com a ficha 7) e, por fim, as propriedades e os resultados.

Registo 6

Registo de apoio — regras do Game of Life e lógica das funções

Prompt (Claude):

> Explica bem as regras do Game of Life e quero que me indiques a logica que devo utilizar nas funções sem me dizeres o codigo, começando pela vizinhos

Resposta do LLM:

(O LLM explicou as regras e a lógica das funções, sem código:)

- **Regras:** vizinhança de Moore (8 células, a própria célula não conta) e regra B3/S23. Na forma compacta, uma célula fica viva se tiver 3 vizinhos vivos, ou se estiver viva e tiver 2.
- **Variante do exercício:** a moldura conta para os vizinhos mas não aplica a regra. No modo síncrono, todas as células leem o mesmo estado antigo. No modo assíncrono, muda uma única célula que possa mudar.
- **`vizinhos`:** aceitar só células normais (`ValueError` nas outras), combinar os deslocamentos $-1, 0, +1$ com `product` e excluir a própria célula. Valores esperados para os testes em $(5,5)$, $(1,1)$, $(1,n)$ e (n,n).
- **`sortear_iniciais`:** gerador próprio `random.Random(semente)`, percurso pela ordem de `celulas_iniciais(n)` e um bit equiprovável guardado como inteiro 0/1.

Registo 7

Registo de apoio — guia para `sortear_iniciais` e `construir_fronteira`

Prompt (Claude):

> dá me agora um guia de como fazer as funções sortear_iniciais e construir_fronteira

Resposta do LLM:

(O LLM deu um guia passo a passo, sem código:)

- **`sortear_iniciais`:** criar um gerador local com `random.Random(semente)` em vez do `random` global, para o sorteio não depender da ordem de execução das células. Percorrer `celulas_iniciais(n)` pela ordem da lista e nunca um `set`. Sortear cada célula com probabilidade 1/2 e converter o resultado para `int`.
- **`construir_fronteira`:** partir de `sortear_iniciais(n, semente)`, usando os parâmetros recebidos e não `N` nem `SEMENTE`. Acrescentar cada célula de

`celulas_finais(n)` com valor 0, de preferência num dicionário novo.

- **Verificações:**
  - $2n+1$ chaves no sorteio e $4n+4$ na fronteira;
  - reprodutibilidade com a mesma semente, mesmo depois de usar o `random` global;
  - nenhuma interseção com as células normais;
  - cantos $(0, n+1)$ e $(n+1, 0)$ com valor 0;
  - união com as normais igual à grelha $(n+2)^2$.
- **A seguir:** ver a moldura sorteada junto ao bloco 3×3, ativar os testes da secção 3 e passar ao simulador (`estado_inicial_concreto` e `passo_concreto`).

Registo 8

Registo de apoio — revisão da geometria e guia para `declarar_estado` e `valor_celula`

Prompt (Claude):

> (código de `sortear_iniciais` e `construir_fronteira`)
>
> Verifica se está tudo feito que é necessário para a geometria de grelha, se sim avancemos para as funções declarar_estado e valor_celula Indica a logica que devo utilizar

Resposta do LLM:

(O LLM reviu o código e deu um guia passo a passo, sem código:)

- **Revisão:** `sortear_iniciais` e `construir_fronteira` estão corretas. O código da geometria está completo. Faltam apenas tarefas que não bloqueiam o resto: ativar os testes da secção 3, os comentários de registo LLM e uma frase sobre o gerador local e sobre `vizinhos` .
- **Papel das funções:** um traço de k estados usa k cópias das variáveis, uma por instante. `declarar_estado(k)` cria a cópia k. `valor_celula` esconde a diferença entre células normais, que são variáveis, e células da moldura, que são constantes.
- **`declarar_estado` :** uma variável `BOOL` por célula normal, para o domínio ficar correto sem restrições $0 \le c \le 1$. O nome inclui i, j e k com separadores, para evitar colisões silenciosas como $(1, 11)$ e $(11, 1)$. Verificar com $n \ge 11$.
- **`valor_celula` :**
  - devolver a variável se a célula for normal;
  - devolver `TRUE()` ou `FALSE()` se for da moldura;
  - lançar exceção se não for nenhuma das duas;

    - devolver sempre uma fórmula booleana, deixando a conversão para número a `proximo_valor`.
- **Simulador independente:** o simulador não deve usar `valor_celula`, para que o teste de concordância consiga detetar erros no modelo simbólico.
- **Verificações:** casos normal, final, inicial viva e morta, e fora da grelha. Teste de integração com `vizinhos` para todas as células normais.
- **Otimização opcional:** somar as vizinhas da moldura como números Python em `proximo_valor`.
- **A seguir:** `init` e `proximo_valor`.

Registo 9

Registo de apoio — revisão do estado simbólico e guia para `init` e `proximo_valor`

Prompt (Claude):

> (código de `declarar_estado` e `valor_celula`)
>
> Guia me agora para fazer as funções inits e proximo_valor

Resposta do LLM:

(O LLM reviu o código e deu um guia passo a passo, sem código:)

- **Revisão:** `declarar_estado` e `valor_celula` estão corretas. Os nomes têm os três índices com separadores e o tipo é `BOOL`. Os três casos estão tratados e o resultado é sempre uma fórmula booleana.
- **`init`:**
    - o estado inicial é o quadrado $(1..3)\times(1..3)$ vivo, porque «$i, j \le 3$» exige as duas condições;
    - é preciso fixar todas as n^2 células normais, com a variável ou a sua negação, e juntar tudo com `And`;
    - se se fixarem só as 9 vivas, `init` descreve 2^{280} estados em vez de um;
    - a moldura não precisa de restrições, e o parâmetro `fronteira` fica só para manter as assinaturas uniformes.
- **Verificar `init`:** a fórmula tem de ser satisfazível e ter uma só solução. Para o confirmar, juntar a condição «alguma célula difere do modelo encontrado», que tem de dar insatisfazível.
- **`proximo_valor`:**
    - devolve, só com as variáveis de `s`, o valor seguinte da célula; a ligação ao estado seguinte `p` é feita pelas funções de transição;
    - obter as vizinhas com `vizinhos`, que também valida a célula;

- somar as vizinhas da moldura num inteiro Python (a «base») e converter cada vizinha normal com `Ite(…, Int(1), Int(0))`;
- a contagem é o `Plus` destes termos com a base;
- por fim, aplicar a regra B3/S23 a essa contagem.

Registo 10

Registo de apoio — revisão de `init` e `proximo_valor` e guia para as relações de transição

Prompt (Claude):

> (código de `init` e `proximo_valor`)
>
> Segue a logica de trans_sincrona, trans_assincrona e trans da forma que tinhas mencionado

Resposta do LLM:

(O LLM reviu o código e deu um guia passo a passo, sem código:)

- **Revisão:** `init` fixa todas as células normais e `proximo_valor` separa as vizinhas constantes das variáveis, com a regra na forma compacta. Único reparo: indicar na docstring de `init` que `fronteira` só existe para uniformizar as assinaturas.
- **Ideia comum:** o valor da regra calcula-se sempre a partir de `s`, nunca de `p`. Sugestão de eficiência: calcular `proximo_valor` uma vez por célula e guardar num dicionário.
- **`trans_sincrona`:** conjunção, para cada célula, de `Iff(p[c], proximo_valor(s, c))`. É determinista, por isso o modo síncrono tem uma única execução. Verificar com um blinker e com a unicidade do sucessor.
- **`trans_assincrona`:**
  - para cada célula, definir `pode_mudar` (a regra dá um valor diferente do atual) e `mudou` (`p` difere de `s`), ambas como negação de um `Iff`;
  - impor que uma célula só muda se podia mudar;
  - impor que o número de células que mudam (soma de `Ite`) é 1 se alguma pode mudar e 0 caso contrário. Esta formulação tem tamanho linear, em vez de uma disjunção com n^4 equivalências, e o estado estável dá o lacete.
  - a interpretação escolhida evita que a propriedade 2 tenha contra-exemplos triviais por o sistema ficar parado.
  - verificar contando os sucessores com o solver, que devem ser tantos quantas as células que podem mudar, e testar um estado estável.

- **`trans`:** escolher a função pelo `modo`, passar o `n` recebido e lançar `ValueError` (com `MODOS`) para valores inválidos. Verificar comparando as fórmulas devolvidas.
- **A seguir:** `erro` e `fabricar_fsm`, e depois o simulador da secção 5.

Registo 11

Registo de apoio — guia para `erro` e `fabricar_fsm`

Prompt (Claude):

> (código de `trans_sincrona`, `trans_assincrona` e `trans`)
>
> Guia me agora para as funções erros e fabricar_fsm

Resposta do LLM:

(O LLM reviu o código e deu um guia passo a passo, sem código:)

- **Revisão:** as três relações de transição estão corretas. Falta confirmar a assíncrona contando os sucessores com o blinker e o bloco 2×2.
- **`erro`:**
  - conjunção da negação de todas as variáveis das células normais;
  - não recebe `fronteira`, porque o enunciado define o erro só com as células normais;
  - o estado de erro não é necessariamente final: as células junto à moldura podem renascer por causa das células iniciais vivas, o que é uma observação útil para a conclusão.
- **Verificar `erro`:** é satisfazível com um único modelo (tudo morto); com `init` é insatisfazível, para não haver «contra-exemplo» com zero passos; um estado com uma célula viva simplifica para `FALSE()`.
- **`fabricar_fsm`:**
  - devolve `(declare, init, trans)` como clausuras que já fixam `fronteira`, `modo` e `n`, como o BMC da ficha 7 espera;
  - validar o `modo` logo no início, para o erro aparecer à criação da FSM e não a meio do BMC;
  - dar às funções interiores nomes diferentes de `init` e `trans`, para evitar recursão infinita;
  - passar sempre `n` e os restantes parâmetros às funções globais;
  - na secção 9, chamar `fabricar_fsm` dentro do ciclo sobre `MODOS`, em vez de escrever um `lambda` com a variável do ciclo.

- **Verificar `fabricar_fsm`:** as fórmulas coincidem com as das funções globais nos dois modos; com n = 20 há 400 variáveis; um modo inválido lança `ValueError`; um «BMC de um passo à mão» no modo síncrono dá um único estado 1, a comparar mais tarde com `passo_concreto`.

- **Fecho da secção 4:** ativar os testes da regra e das transições, acrescentar uma frase sobre o determinismo do modo síncrono e passar ao simulador da secção 5.

Registo 12

Registo de apoio — guia para `estado_inicial_concreto` e `passo_concreto`

Prompt (Claude):

> explica me a melhor forma de fazer estado_inicial_concerto e passo_concreto

Resposta do LLM:

(O LLM deu um guia passo a passo, sem código:)

- **Princípio orientador:** o simulador é um oráculo independente. Pode usar a geometria (`celulas_normais`, `vizinhos`, `construir_fronteira`), mas não `valor_celula`, `init`, `proximo_valor` nem `trans_*`, e nada do pySMT. A grelha é um dicionário célula normal → 0/1, com as mesmas chaves que `declarar_estado`.
- **`estado_inicial_concreto`:** 1 para $i, j \le 3$ e 0 nas restantes, escrito à parte de `init` para o `test_estado_inicial` comparar as duas.
- **`passo_concreto`:**
  - uma função auxiliar (`aplicar_regra`) que aplica a regra a todas as células, reutilizável em `traco_e_valido` e registada nas «funções acrescentadas»;
  - ler o valor de cada vizinha na grelha ou, se não for normal, na fronteira;
  - escrever a regra na forma da tabela, diferente da forma compacta de `proximo_valor`, para um erro de raciocínio não se repetir nos dois lados;
  - calcular todos os valores a partir da grelha antiga, num dicionário novo;
  - no modo síncrono, devolver o resultado da regra;
  - no modo assíncrono, listar as células instáveis pela ordem de `celulas_normais(n)`, devolver uma cópia se não houver nenhuma (lacete) e, caso contrário, trocar só uma, escolhida com o `rng`;
  - o gerador é criado por quem chama, uma vez por simulação, e nunca se usa o `random` global; sem `rng` no modo assíncrono, lançar `ValueError` em vez de criar um gerador por omissão; no modo síncrono, o `rng` é ignorado;

    - lançar `ValueError` para modos inválidos e nunca alterar a grelha recebida.
- **Verificações:**
    - padrões no interior da grelha: blinker, bloco 2×2 e célula isolada;
    - o blinker assíncrono com várias sementes tem de dar exatamente os 4 sucessores;
    - reprodutibilidade com a mesma semente;
    - a grelha de entrada não muda;
    - concordância com `trans_sincrona` em grelhas aleatórias e no estado inicial.
- **A seguir:** simular o modo síncrono até repetir um estado, para saber se o erro é alcançado, o transiente e o período do ciclo (que orientam o `K_MAX`) e que células ficam constantes. Depois, `traco_e_valido` e `apresentar_grelha`.

Registo 13

Registo de apoio — revisão do simulador e guia para `traco_e_valido` e `apresentar_grelha`

Prompt (Claude):

> (código de `estado_inicial_concreto` e `passo_concreto`)
>
> explica me agora então como fazer traco_e_valido e apresentar_grelha

Resposta do LLM:

(O LLM reviu o código e deu um guia passo a passo, sem código:)

- **Revisão:**
    - `estado_inicial_concreto` está correta;
    - **erro em `passo_concreto`:** no modo síncrono, `nova = grelha` não copia o dicionário. As contagens passam a ler valores já atualizados (o erro clássico do Game of Life) e a grelha recebida é alterada. Correção: ler e escrever em dicionários diferentes;
    - com `rng=None`, criar o gerador dentro da função repete sempre a mesma escolha. Deve lançar-se `ValueError`;
    - sugestão: tirar a regra para uma função própria, `aplicar_regra`, reutilizável em `traco_e_valido` e escrita na forma da tabela.
- **`traco_e_valido`:**
    - devolve uma lista de mensagens de erro, vazia se o traço for válido, para indicar onde e porquê falha;
    - lança `ValueError` para modos inválidos e trata o traço vazio como erro;
    - normaliza os valores para 0/1, para aceitar traços do solver com `True`/`False`, e verifica as chaves e os valores de cada grelha;

    - o primeiro estado tem de ser `estado_inicial_concreto(n)`;
    - no modo síncrono, cada estado tem de ser igual à regra aplicada ao anterior;
    - no modo assíncrono, sem instáveis não há mudanças (lacete), e com instáveis muda exatamente uma, que é instável;
    - para validar um lacete, quem chama acrescenta ao fim do traço uma cópia do estado onde o lacete começa.
- **Verificar `traco_e_valido`:** traços simulados válidos nos dois modos; uma célula trocada; troca de modo; estado inicial errado; lacete indevido; valores `True` / `False`.
- **`apresentar_grelha`:**
    - devolve um `DataFrame` sem usar `mo`, para continuar a ser uma `@app.function`;
    - usa rótulos de 1 a n, ou de 0 a $n+1$ com a moldura;
    - lê o valor da grelha ou da fronteira e mostra símbolos em vez de 0/1, distinguindo a moldura;
    - a legenda vai por cima da tabela.
- **A seguir:** a célula interativa da secção 5 e a simulação síncrona até um estado se repetir, antes do BMC.

Registo 14

Registo de apoio — revisão de `traco_e_valido` e `apresentar_grelha` e guia para a célula interativa

Prompt (Claude):

> (código de `traco_e_valido` e `apresentar_grelha`)

Resposta do LLM:

(O LLM reviu o código:)

- **`traco_e_valido`:** está correta, com os três casos do modo assíncrono bem encadeados e mensagens que indicam o passo, a célula e o valor. Há dois pormenores, nenhum deles bloqueante:
    - validar os valores em $\{0, 1\}$ antes de os converter com `int`, para não aceitar em silêncio valores como `0.7`;
    - juntar as células erradas de cada passo síncrono numa só mensagem, para o resultado ficar legível.
- **`apresentar_grelha`:** está correta. A legenda deve aparecer também por cima da tabela. Se a tabela interativa do Marimo cortar a grelha em páginas, pode usar-se `mo.ui.table` com o tamanho de página igual ao número de linhas, ou `mo.Html`.

- **Estado da secção 5:** todas as funções estão feitas. Falta testar `traco_e_valido` com o quadro do guia, fazer a célula interativa e simular o modo síncrono até um estado se repetir.

Prompt (Claude):

> Guia me então na célula interativa

Resposta do LLM:

(O LLM explicou como construir a simulação interativa:)

- `mo.md` **:** mostra texto em Markdown como saída da célula. A célula com «*Simulação por implementar.*» é só um marcador, e é substituída.
- **Duas células:** no Marimo, o valor de um controlo só pode ser lido numa célula diferente daquela onde o controlo é criado.
  - **Célula 1:** cria um `mo.ui.slider` com o número de passos (até 200, para chegar ao estado fixo do modo síncrono) e um `mo.ui.dropdown` com `MODOS`, mostrados lado a lado com `mo.hstack`.
  - **Célula 2:** constrói a fronteira e um gerador `random.Random(SEMENTE)`, parte de `estado_inicial_concreto` e aplica `passo_concreto` tantas vezes quanto o valor do slider. Mostra com `mo.vstack` um texto com o passo, o modo, o número de células vivas e a legenda, seguido da tabela de `apresentar_grelha`.
- **Reatividade:** ao mexer nos controlos, o Marimo volta a correr a célula 2 automaticamente.
- **Nomes com** `_` **:** ficam locais à célula e evitam conflitos com nomes definidos noutras células.
- **Reprodutibilidade:** a simulação recomeça sempre do início com a mesma semente, por isso o mesmo número de passos dá sempre a mesma grelha, também no modo assíncrono.

Prompt (Claude):

> Podes me dar o código da célula interativa, não sei como fazer este tipo de funções

Resposta do LLM:

(O LLM escreveu o código da simulação interativa, dividido em três células:)

- **Célula A — controlos:** a constante `PASSOS_SIMULACAO = 200` e quatro controlos: um `mo.ui.dropdown` para o modo, um `mo.ui.slider` para o passo, um `mo.ui.number` para a semente do escalonador (só no modo assíncrono) e um `mo.ui.checkbox` para mostrar a moldura.

- **Célula B — simulação:** lê o modo e a semente, mas não o slider. Constrói a fronteira com `SEMENTE` e cria um único gerador para a simulação toda. Calcula o traço completo com `passo_concreto` e valida-o com `traco_e_valido`.
- **Célula C — visualização:** lê o slider e mostra o estado escolhido. Por cima da tabela indica o número de células vivas, assinala o estado de erro e o estado estável, mostra a célula que mudou (no modo assíncrono), o resultado da validação e a legenda. A tabela é mostrada com `mo.Html`, para não ficar paginada.
- **Porque são três células:** o valor de um controlo só pode ser lido numa célula diferente daquela onde é criado. Com a simulação separada da visualização, mover o slider não recalcula o traço.

Link da conversa: (colar aqui o link da conversa com o Claude)